

TECHNIQUES AND TOOLS FOR TAMING TANGLED TWISTED TRAINS OF THOUGHT

A Talk by Tim Caswell

[<tim@creationix.com>](mailto:tim@creationix.com)

WHAT MAKES NODE FAST?

- Node.js is fast by design.
- Never blocking on I/O means less threads.
- This means YOU handle scheduling.



HELLO I/O (IN RUBY)

Open a file

```
file = File.new("readfile.rb", "r")
```

Read the file

```
while (line = file.gets)
```

```
  # Do something with line
```

```
end
```

Close the file

```
file.close
```

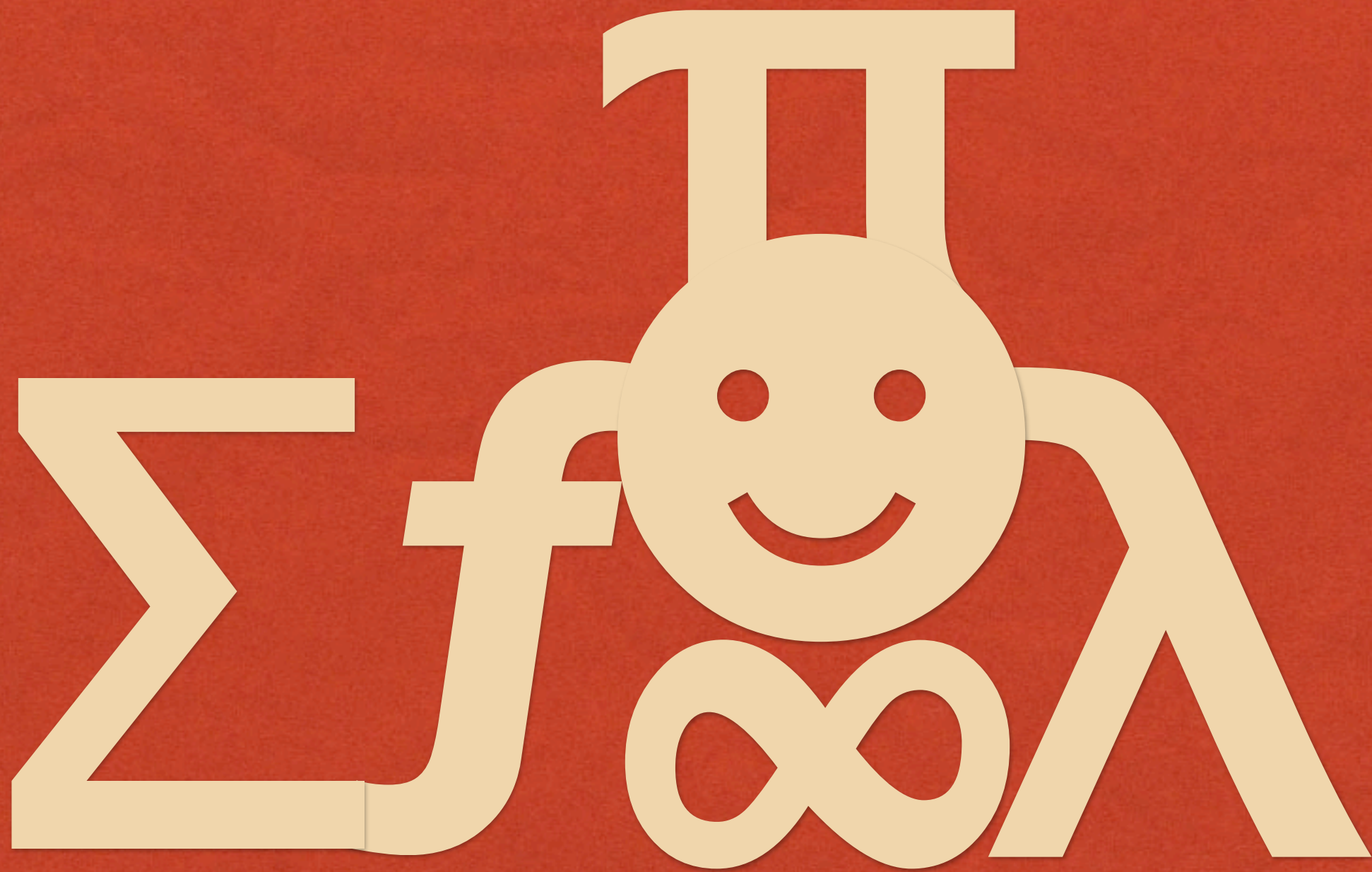

HELLO I/O (IN NODE)

```
// This is a "simple" naive implementation
fs.open('readfile.js', 'r', function (err, fd) {
  var length = 1024, position = 0,
      chunk = new Buffer(length);
  function onRead(err, bytesRead) {
    if (bytesRead) {
      chunk.length = bytesRead;
      // Do something with chunk
      position += bytesRead;
      readChunk();
    } else {
      fs.close(fd);
    }
  }
  function readChunk() {
    fs.read(fd, chunk, 0, length, position, onRead);
  }
  readChunk();
});
```


A firefighter in a black shirt and yellow pants stands with hands on hips, looking at a large fire burning in a building. A long, bright orange stream of water from a hose arcs across the frame from the firefighter towards the fire. Thick black smoke billows from the fire. The scene is outdoors with trees and a grassy area in the foreground.

**It's not as bad
as it looks,
I promise.**

Let's learn some tricks!



Tangled Twisted Train of Thought



First Class Functions

λ FIRST CLASS FUNCTIONS

- JavaScript is a very simple, but often misunderstood language.
- The secret to unlocking it's potential is understanding it's functions.
- A function is an object that can be passed around with an attached closure.
- Functions are NOT bound to objects.

λ FIRST CLASS FUNCTIONS

```
var Lane = {  
  name: "Lane the Lambda",  
  description: function () {  
    return "A person named " + this.name;  
  }  
};
```

```
Lane.description();  
// A person named Lane the Lambda
```


λ FIRST CLASS FUNCTIONS

```
var Fred = {  
  name: "Fred the Functor",  
  descr: Lane.description  
};  
  
Fred.descr();  
// A person named Fred the Functor
```


λ FIRST CLASS FUNCTIONS

```
Lane.description.call({  
  name: "Zed the Zetabyte"  
});  
// A person named Zed the Zetabyte
```


λ FIRST CLASS FUNCTIONS

```
var descr = Lane.description;  
descr();  
// A person named undefined
```


λ FIRST CLASS FUNCTIONS

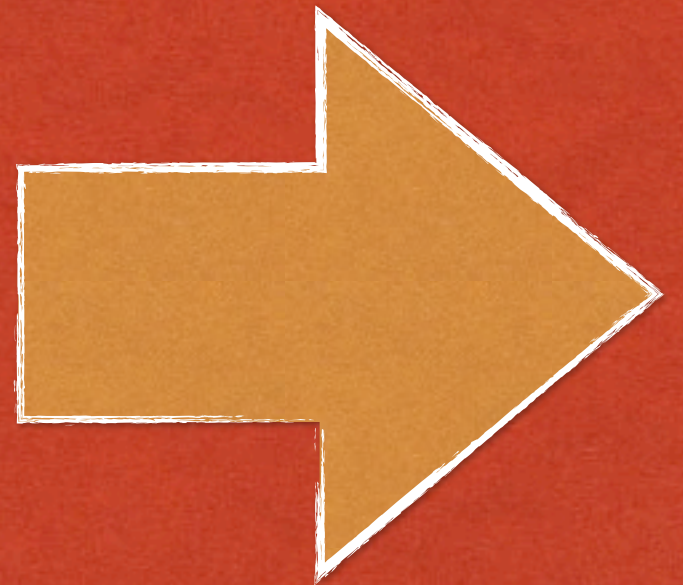
```
function makeClosure(name) {  
  return function description() {  
    return "A person named " + name;  
  }  
}
```

```
var description =  
  makeClosure('Cloe the Closure');
```

```
description();  
// A person named Cloe the Closure
```




First Class
Functions



A large, stylized, lowercase letter 'f' in a light beige or cream color. The letter is bold and has a modern, slightly rounded design. It is positioned on the left side of the image, set against a solid, textured red background.

Function Composition



FUNCTION COMPOSITION

```
fs.readFile(filename, callback) {
```

Open File

Read Contents

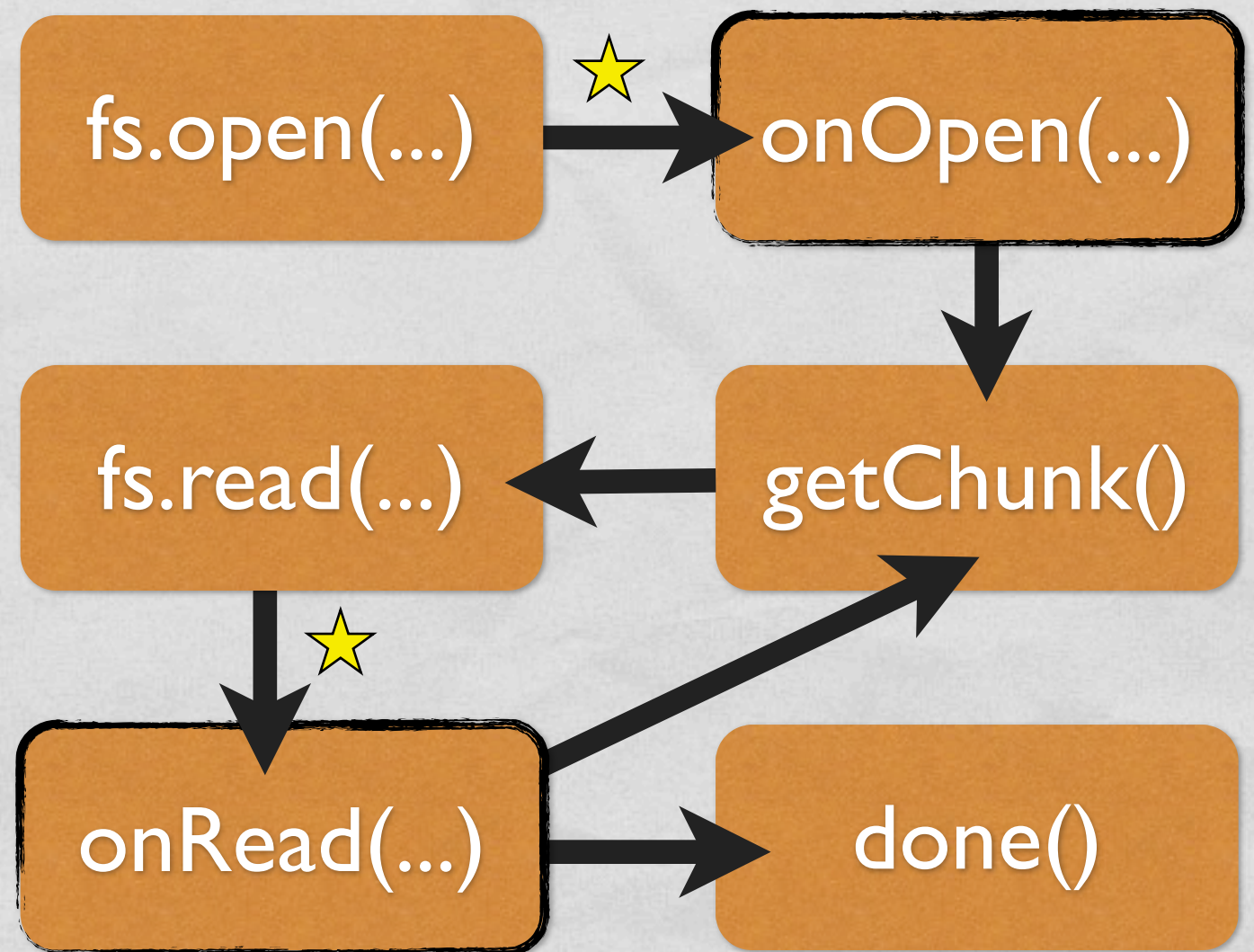
Close File

```
};
```


f FUNCTION COMPOSITION

- Several small functions make one large one.
- Star means call is via the event loop.
- Black border means the function is wrapped.

```
fs.readFile(...) {
```



```
};
```


f FUNCTION COMPOSITION

```
var fs = require('fs');

// Easy error handling for async handlers
function wrap(fn, callback) {
  return function wrapper(err, result) {
    if (err) return callback(err);
    try {
      fn(result);
    } catch (err) {
      callback(err);
    }
  }
}
```


f FUNCTION COMPOSITION

```
function mergeBuffers(buffers) {  
  if (buffers.length === 0) return new Buffer(0);  
  if (buffers.length === 1) return buffers[0];  
  var total = 0, offset = 0;  
  buffers.forEach(function (chunk) {  
    total += chunk.length;  
  });  
  var buffer = new Buffer(total);  
  buffers.forEach(function (chunk) {  
    chunk.copy(buffer, offset);  
    offset += chunk.length;  
  });  
  return buffer;  
}
```


f FUNCTION COMPOSITION

```
function readFile(filename, callback) {  
  var result = [], fd,  
      chunkSize = 40 * 1024,  
      position = 0, buffer;  
  
  var onOpen = wrap(function onOpen(descriptor) {...});  
  
  var onRead = wrap(function onRead(bytesRead) {...});  
  
  function getChunk() {...}  
  
  function done() {...}  
  
  fs.open(filename, 'r', onOpen);  
}
```




FUNCTION COMPOSITION

```
var onOpen = wrap(  
  function onOpen(descriptor) {  
    fd = descriptor;  
    getChunk();  
  },  
  callback  
);
```


f FUNCTION COMPOSITION

```
var onRead = wrap(  
  function onRead(bytesRead) {  
    if (!bytesRead) return done();  
    if (bytesRead < buffer.length) {  
      var chunk = new Buffer(bytesRead);  
      buffer.copy(chunk, 0, 0, bytesRead);  
      buffer = chunk;  
    }  
    result.push(buffer);  
    position += bytesRead;  
    getChunk();  
  },  
  callback  
);
```


f FUNCTION COMPOSITION

```
function getChunk() {  
  buffer = new Buffer(chunkSize);  
  fs.read(fd, buffer, 0,  
          chunkSize, position, onRead);  
}
```




FUNCTION COMPOSITION

```
function done() {  
  fs.close(fd);  
  callback(null, mergeBuffers(result));  
}
```

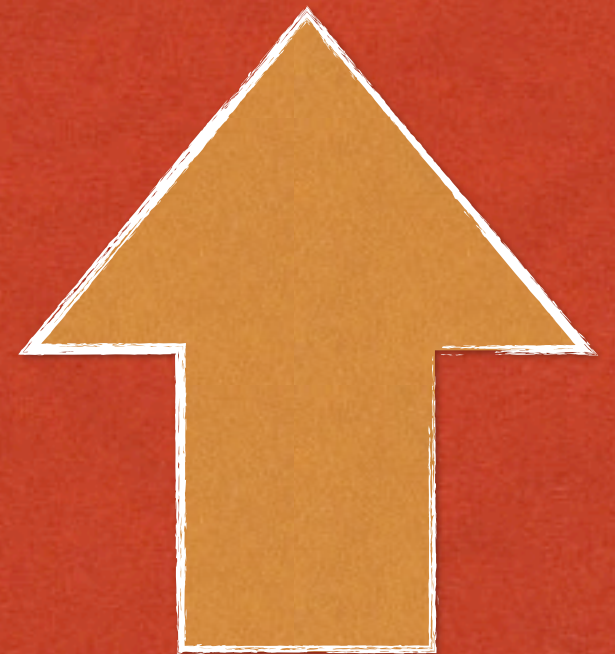

f FUNCTION COMPOSITION

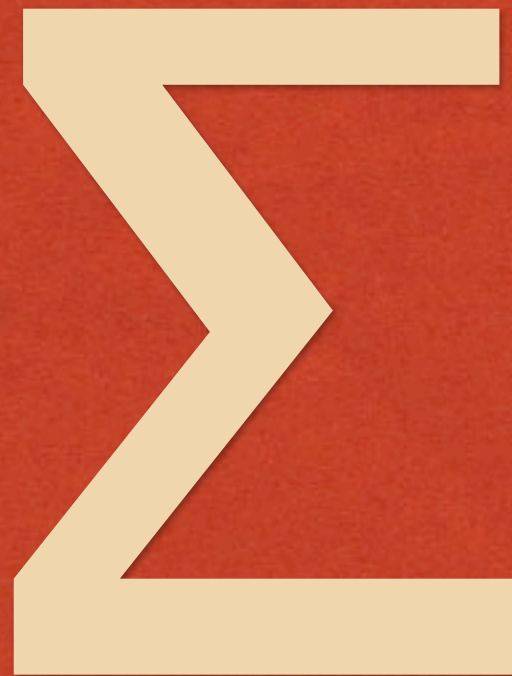
Fit the abstraction to your problem
using function composition.

```
// If you just want the contents of a file...  
fs.readFile('readfile.js', function (err, buffer) {  
  if (err) throw err;  
  // File is read and we're done.  
});  
// The read function is doing it's thing...
```


f

Function Composition





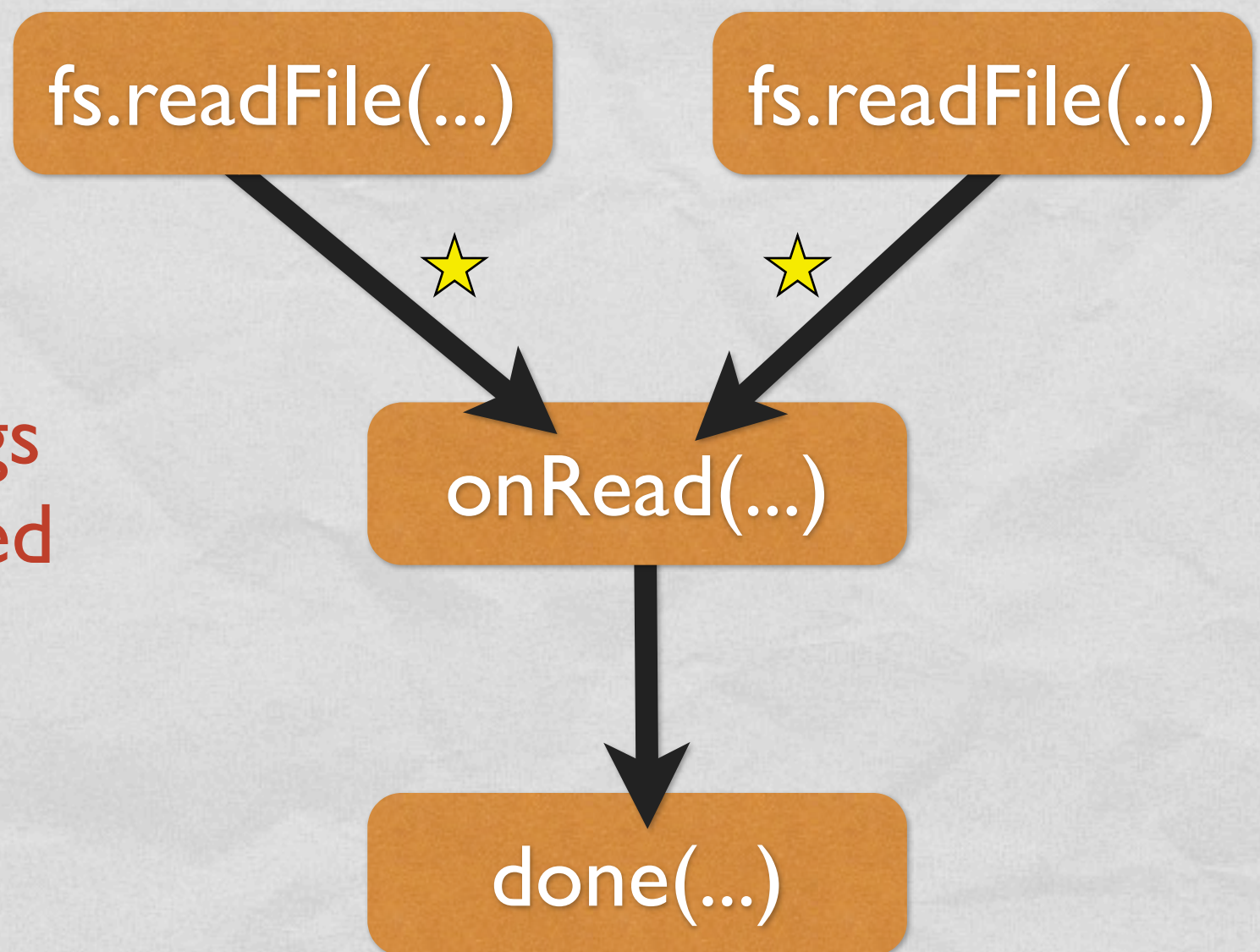
Callback Counters

Σ CALLBACK COUNTERS

- The true power in non-blocking code is parallel I/O made easy.
- You can do other things while waiting.
- You can wait on more than one thing at a time.
- Organize your logic into chunks of serial actions, and then run those chunks in parallel.

Σ CALLBACK COUNTERS

- Sometimes you want to do two async things at once and be notified when both are done.



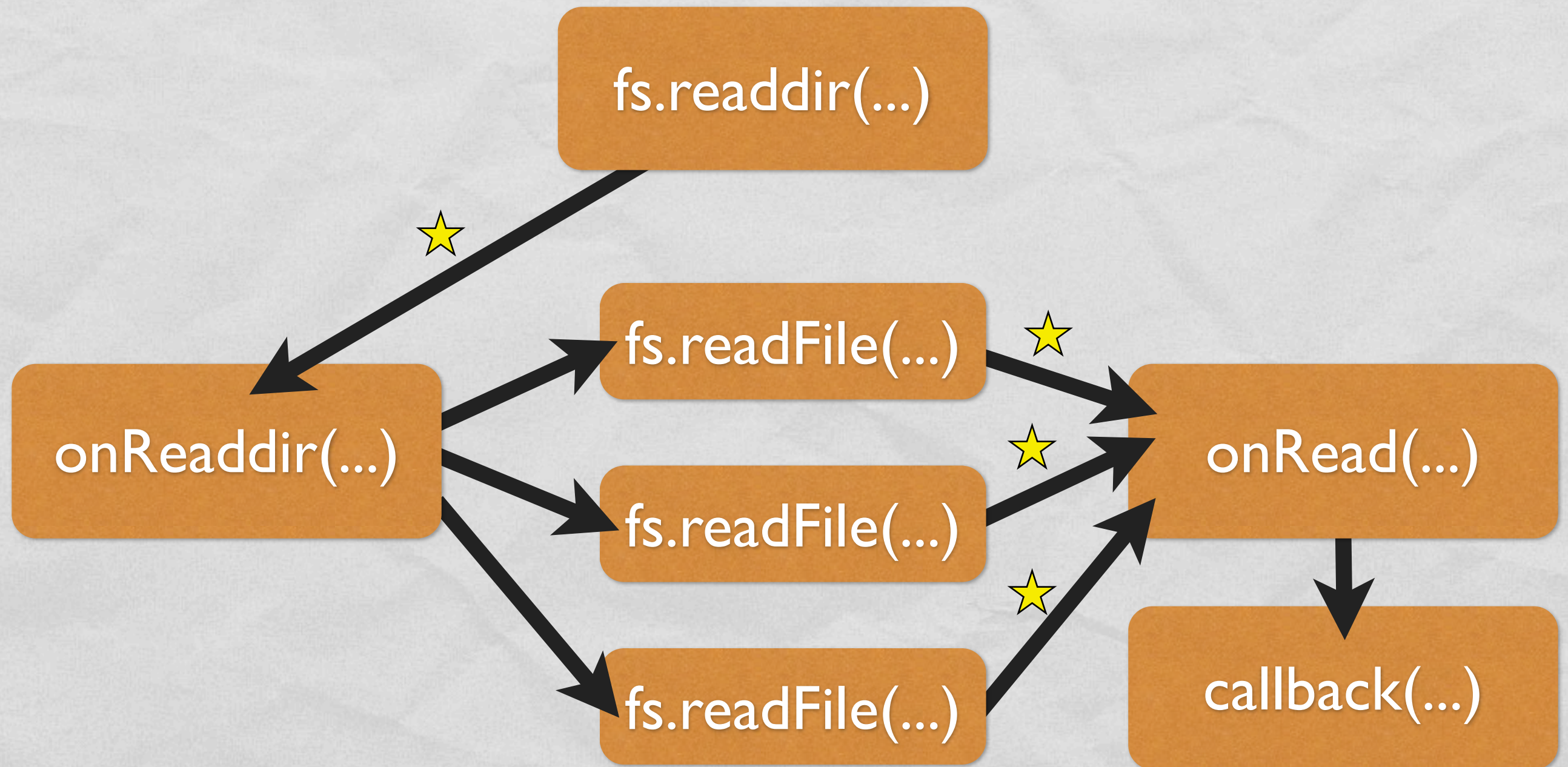
Σ CALLBACK COUNTERS

```
var counter = 2;  
fs.readFile(__filename, onRead);  
fs.readFile("/etc/passwd", onRead);
```

```
function onRead(err, content) {  
    if (err) throw err;  
    // Do something with content  
    counter--;  
    if (counter === 0) done();  
}
```

```
function done() {  
    // Now both are done  
}
```


Σ CALLBACK COUNTERS

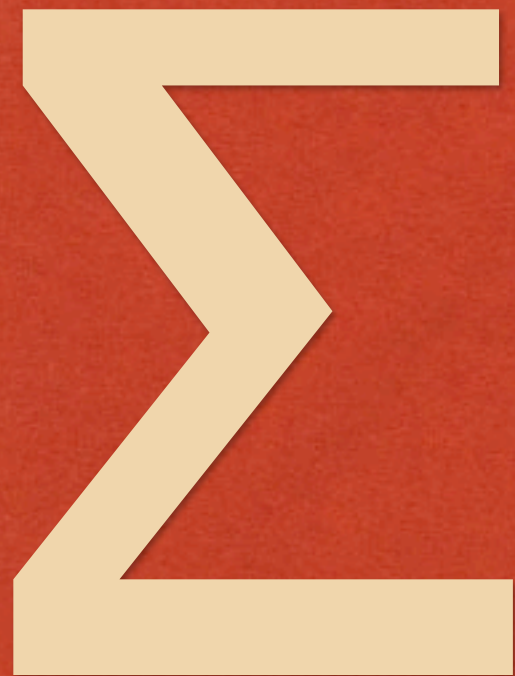


Σ CALLBACK COUNTERS

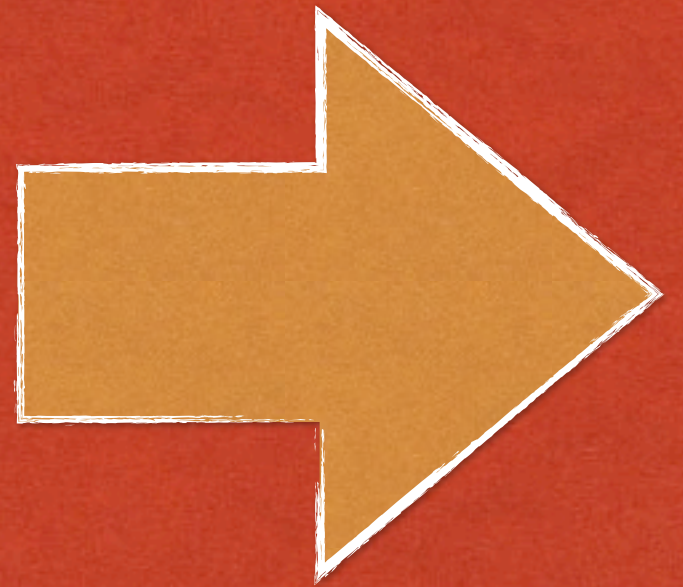
```
function loadDir(directory, callback) {
  fs.readdir(directory, function onReaddir(err, files) {
    if (err) return callback(err);
    var count, results = {};
    files = files.filter(function (filename) {
      return filename[0] !== '.';
    });
    count = files.length;
    files.forEach(function (filename) {
      var path = directory + "/" + filename;
      fs.readFile(path, function onRead(err, data) {
        if (err) return callback(err);
        results[filename] = data;
        if (--count === 0) callback(null, results);
      });
    });
    if (count === 0) callback(null, results);
  });
}
```

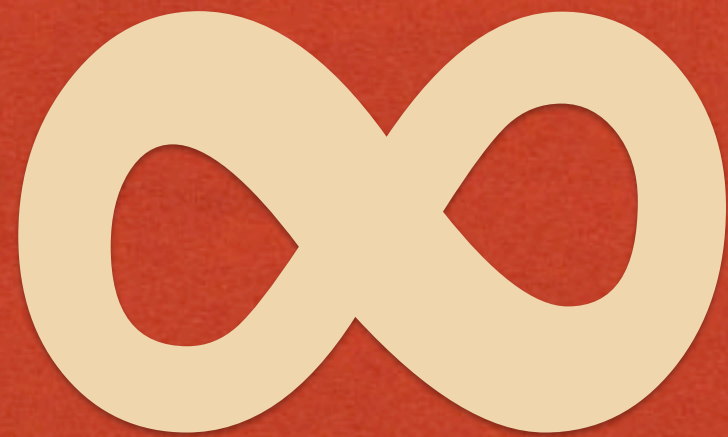

Σ CALLBACK COUNTERS

```
function loadDir(directory, callback) {  
  fs.readdir(directory, function onReaddir(err, files) {  
    //...  
    var count = files.length;  
    files.forEach(function (filename) {  
      //...  
      fs.readFile(path, function onRead(err, data) {  
        //...  
        if (--count === 0) callback(null, results);  
      });  
    });  
    if (count === 0) callback(null, results);  
  });  
}
```

Callback Counters





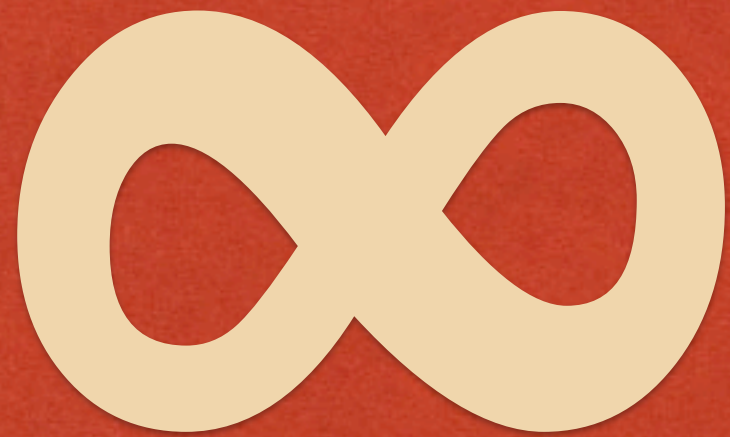
Event
Loops

∞ EVENT LOOPS

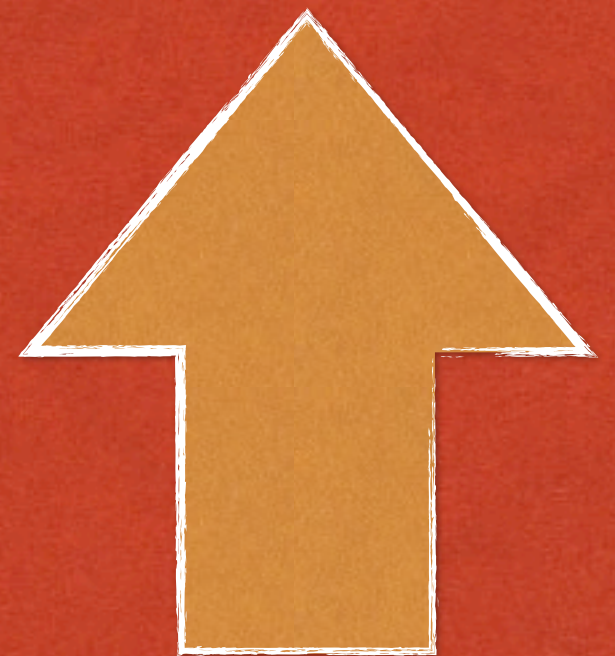
- Plain callbacks are great for things that will eventually return or error out.
- But what about things that just happen sometimes or never at all.
- These general callbacks are great as events.
- Events are super powerful and flexible since the listener is loosely coupled to the emitter.

∞ EVENT LOOPS

```
fs.lineReader('readfile.js', function (err, file) {  
  if (err) throw err;  
  file.on('line', function (line) {  
    // Do something with line  
  });  
  file.on('end', function () {  
    // File is closed and we're done  
  });  
  file.on('error', function (err) {  
    throw err;  
  });  
});
```

Event
Loops





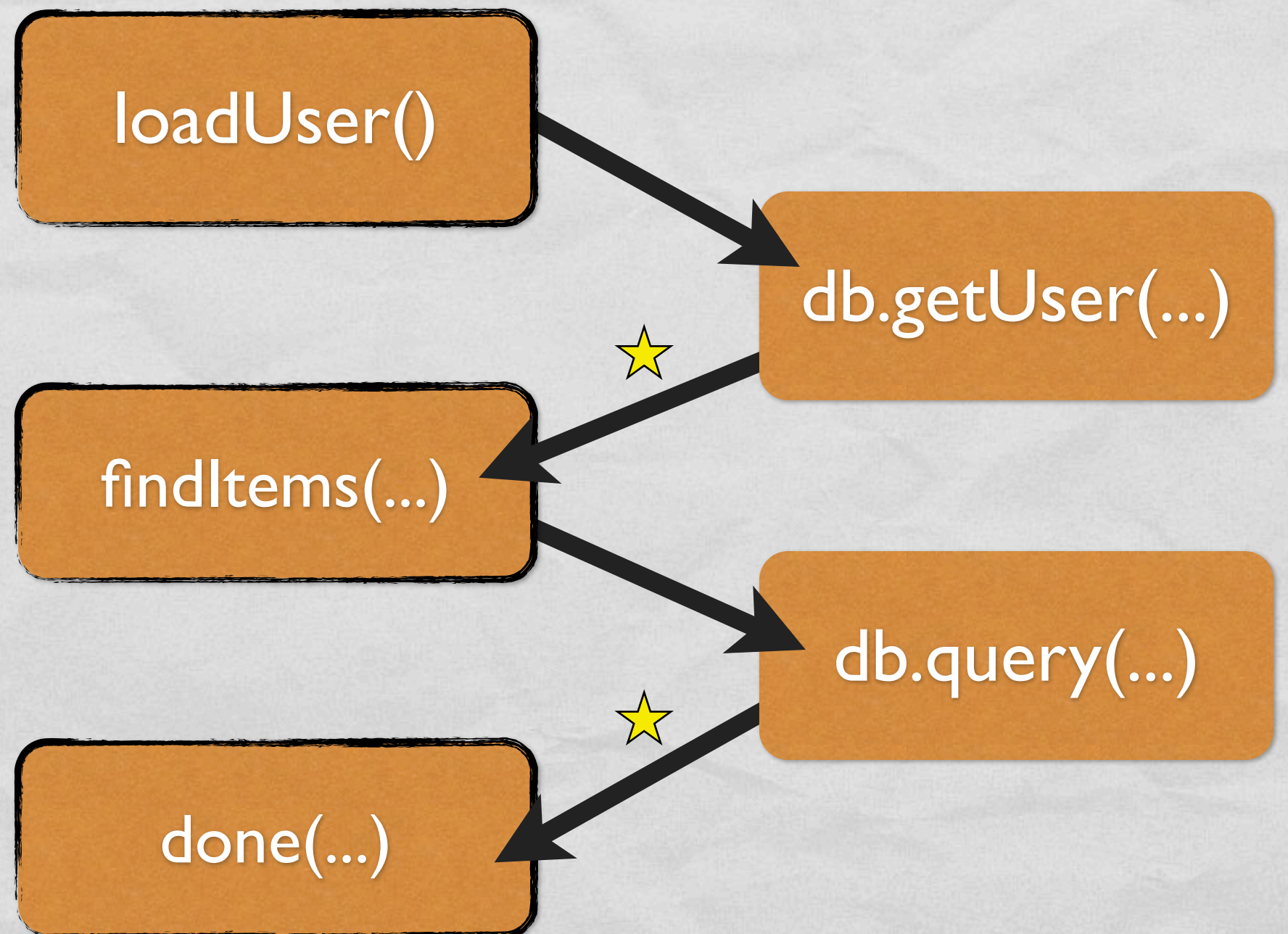
Easy as Pie
Libraries

π EASY AS PIE LIBRARIES

- Step - <http://github.com/creationix/step>
 - Based on node's `callback(err, value)` style.
 - Can handle serial, parallel, and grouped actions.
 - Adds exception handling.
- Promised-IO - <http://github.com/kriszyp/promised-io>
 - Uses the promise abstraction with node's APIs

π EASY AS PIE LIBRARIES

- Serial chains where one action can't happen till the previous action finishes is a common use case.



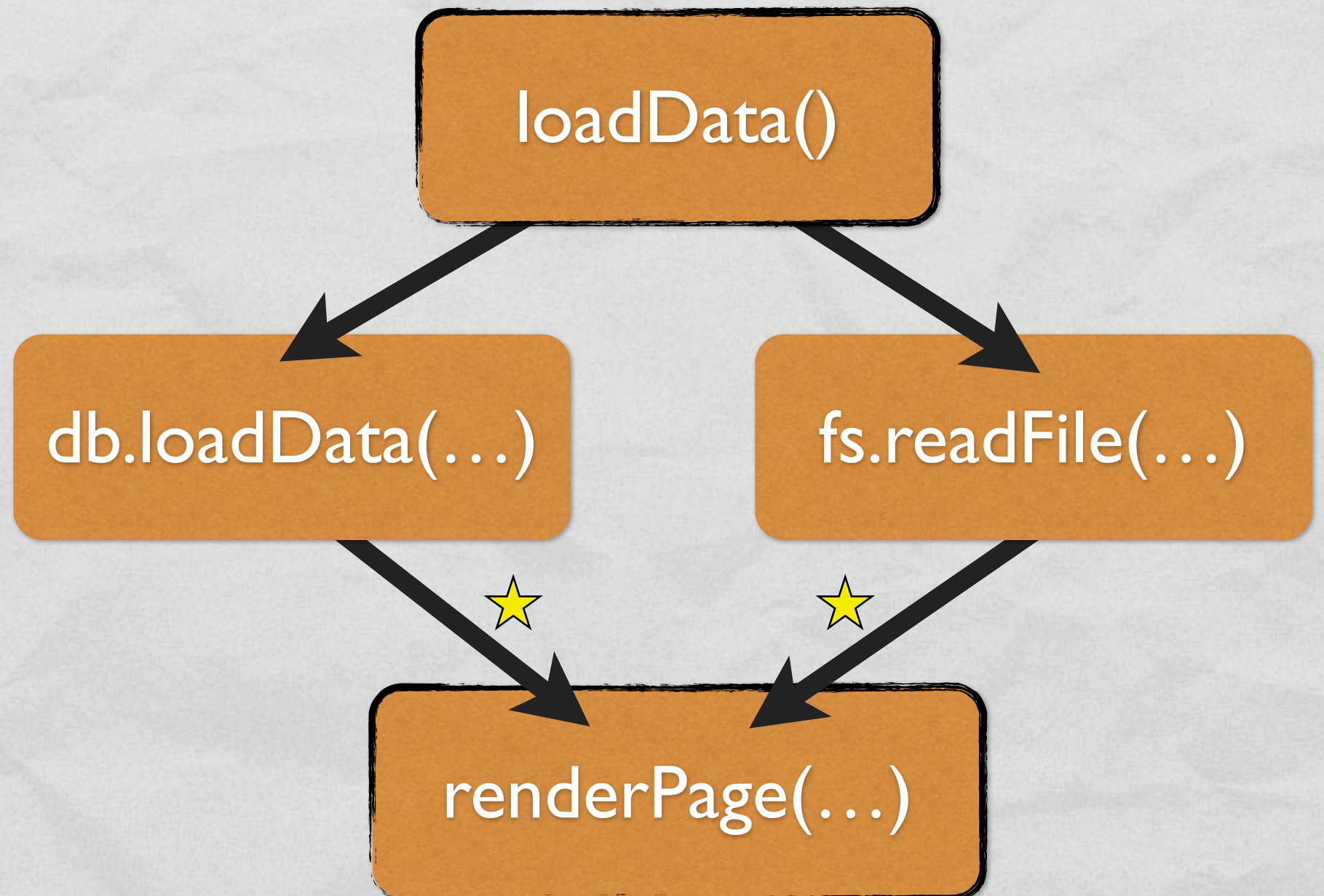
π EASY AS PIE LIBRARIES

Step(

```
function loadUser() {  
  db.getUser(user_id, this);  
},  
function findItems(err, user) {  
  if (err) throw err;  
  var sql = "SELECT * FROM store WHERE type=?";  
  db.query(sql, user.favoriteType, this);  
},  
function done(err, items) {  
  if (err) throw err;  
  // Do something with items  
}  
);
```


π EASY AS PIE LIBRARIES

- Sometimes you want to do a couple things in parallel and be notified when both are done.

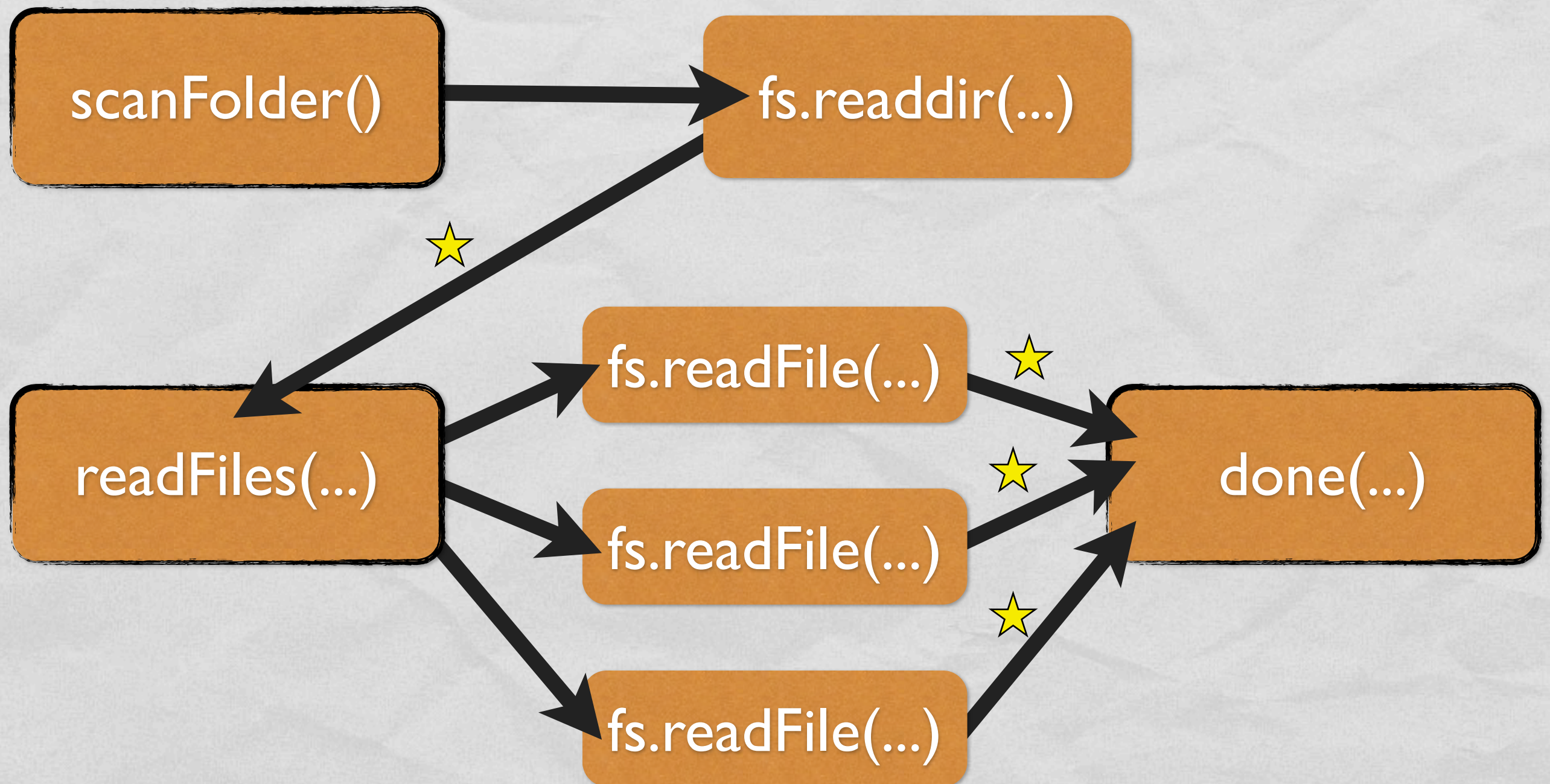


π EASY AS PIE LIBRARIES

Step(

```
function loadData() {  
  db.loadData({some: parameters}, this.parallel());  
  fs.readFile("staticContent.html", this.parallel());  
},  
function renderPage(err, dbResults, fileContents) {  
  if (err) throw err;  
  // Render page  
}  
)
```


π EASY AS PIE LIBRARIES



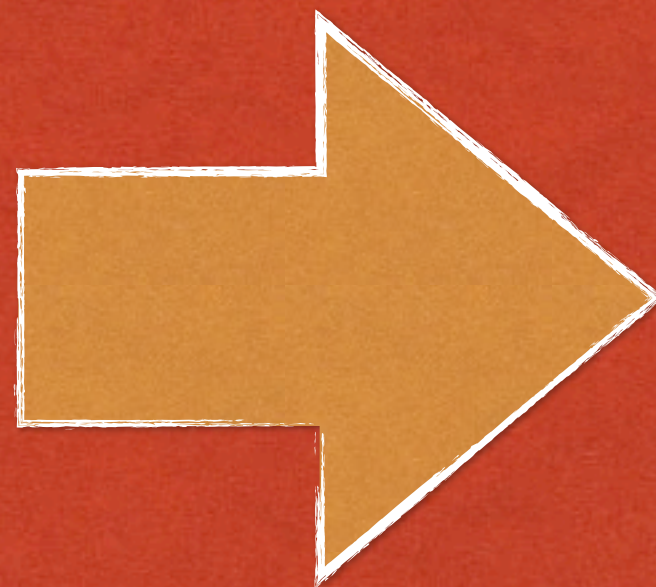
π EASY AS PIE LIBRARIES

Step(

```
function scanFolder() {  
  fs.readdir(__dirname, this);  
},  
function readFiles(err, filenames) {  
  if (err) throw err;  
  var group = this.group();  
  filenames.forEach(function (filename) {  
    fs.readFile(filename, group());  
  });  
},  
function done(err, contents) {  
  if (err) throw err;  
  // Now we have the contents of all the files.  
}  
)
```




Easy as Pie
Libraries





tim@creationix.com

<http://howtonode.org/>

<http://github.com/creationix>